Food Delivery Platform

Technical Specification & 12-Week Development Roadmap

TECHNICAL IN-DEPTH REPORT

Stack: Kotlin (Android) + Python FastAPI (Backend)
Prepared By: Development Team Lead
Date: May 2026 | Version 1.0
CONFIDENTIAL — DEVELOPMENT TEAM USE

Project Name: Food Delivery Platform (Android)

Report Type: Technical In-Depth Development Report

Prepared By: Development Team Lead

Date: May 2026

Timeline: 12 Weeks (3 Months)

Target Platform: Android (Native Kotlin)

---

1. Project Overview

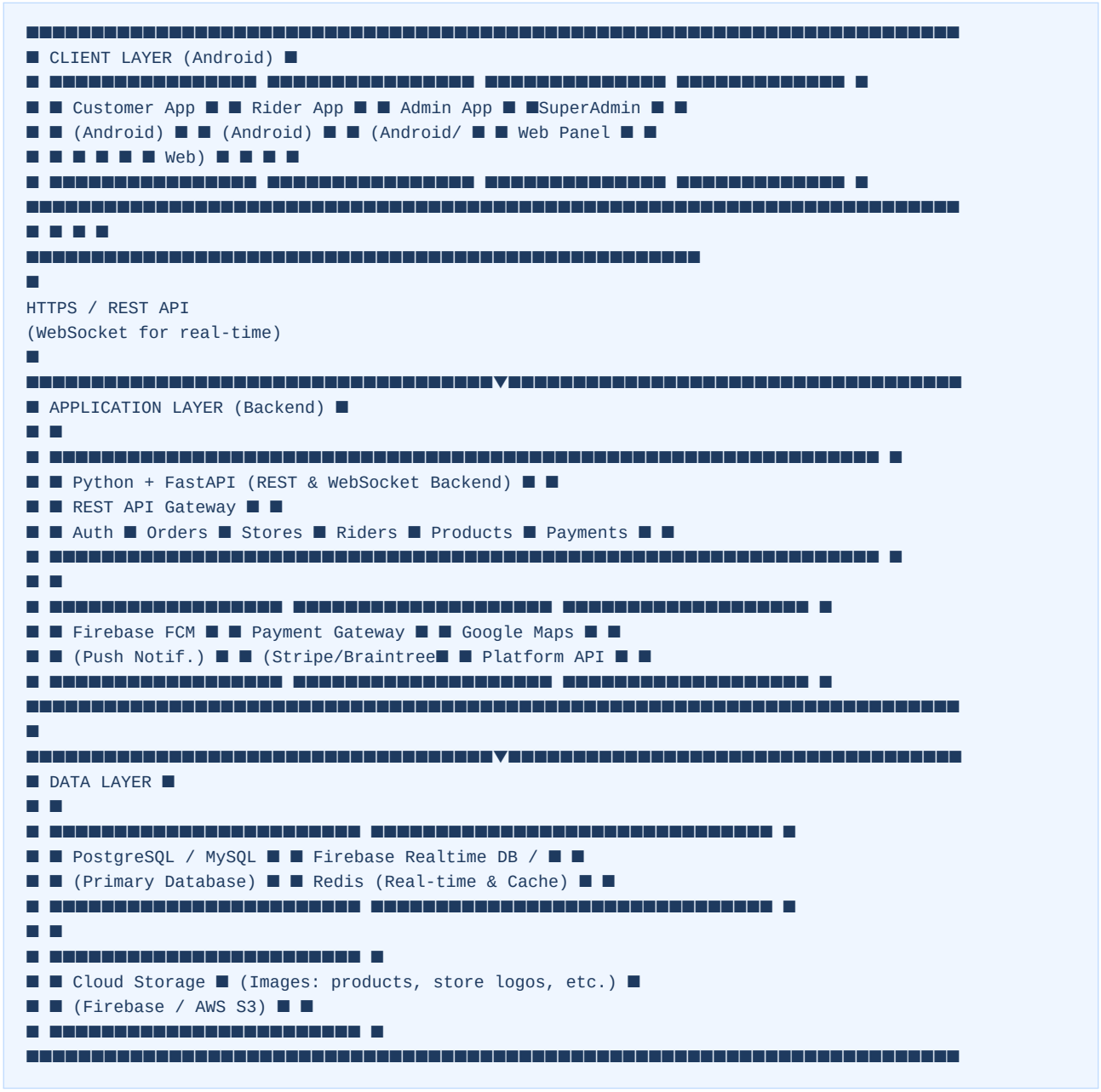
This document serves as a complete technical specification and development roadmap for a multi-role food delivery Android application. The system supports four distinct user roles **Customer**, **Rider**, **Store Admin**, and **Super Admin** each with its own dedicated application interface and backend access scope.

The platform is architected as a **multi-tenant food delivery ecosystem**, enabling multiple restaurant stores to operate under one platform umbrella, with full real-time order management, GPS-tracked deliveries, and integrated online payment.

---

2. System Architecture

2.1 High-Level Architecture



2.2 Architecture Style

- **Backend:** Python + FastAPI (RESTful API + WebSocket support for real-time features)
- **Frontend:** Android Native (Kotlin with Jetpack Compose / XML Layouts)
- **Authentication:** JWT (JSON Web Tokens) with role-based access control (RBAC) via `python-jose`
- **Real-time Communication:** Firebase Realtime Database + FastAPI WebSocket for live order tracking
- **File Storage:** Firebase Storage or AWS S3 for product images and store assets

---

3. Technology Stack

Layer	Technology	Justification
Android App	**Kotlin (Native Android)**	Official Google-recommended language; best performance, full Android SDK access
UI Framework	**Jetpack Compose**	Modern declarative Android UI — less boilerplate, richer animations
Backend API	**Python + FastAPI**	High performance async Python API; clean auto-generated OpenAPI docs; async WebSocket support
ORM	**SQLAlchemy + Alembic**	Pythonic ORM with migrations for PostgreSQL schema management
Database	**PostgreSQL**	Relational, reliable, ACID-compliant for transactional data
Real-time DB	**Firebase Realtime DB**	Low-latency for rider location & order status updates
Authentication	**JWT via python-jose**	Secure token-based auth; RBAC middleware per route
Push Notifications	**Firebase Cloud Messaging (FCM)**	Industry standard for Android push notifications
Payment Gateway	**Stripe**	Best Python SDK support; sandbox mode; 3DS card security
Maps & Routing	**Google Maps SDK + Directions API**	Best-in-class for Android GPS tracking and navigation
HTTP Client (Android)	**Retrofit2 + OkHttp**	Type-safe REST client for Kotlin; widely used
DI (Android)	**Hilt (Dagger)**	Kotlin-first dependency injection
Cloud Storage	**Firebase Storage**	Easy integration with Firebase ecosystem
Async (Android)	**Kotlin Coroutines + Flow**	Native Kotlin concurrency for async API calls & real-time streams
Hosting/Deployment	**AWS EC2 / DigitalOcean / Railway**	Scalable, production-ready cloud hosting
Admin Panel	**React.js (Web)**	Lightweight Super Admin web dashboard
Version Control	**Git + GitHub**	Standard

---

4. User Roles & Module Breakdown

4.1 Customer Module

4.1.1 Authentication & Profile

- Register with email/phone + password

- Login with JWT token issuance
- Social login (optional Google OAuth)
- Profile management: name, address book (multiple addresses), phone
- Password reset via OTP/email

4.1.2 Store Discovery

- List all active stores on the platform (paginated)
- Store card: name, logo, category, rating, estimated delivery time
- Search & filter by category (e.g., fast food, desserts, beverages)
- Store detail view: banner, description, operating hours, menu categories

4.1.3 Product & Cart

- Browse products by category within a store
- Product detail: image, name, description, price, availability toggle
- Add to cart with quantity control
- Cart summary: items, quantities, subtotal, delivery fee, total
- Cart persistence (stored locally and synced)
- Cross-store cart warning (orders per store only)

4.1.4 Checkout & Payment

- Confirm delivery address (from address book or new entry)
- Map pin drop to confirm exact delivery location
- Select payment method:
- **Online (Card):** Stripe/Braintree payment form, card tokenization, 3DS support
- **Cash on Delivery (COD):** No upfront payment, pay to rider at door
- Order confirmation screen with order summary and estimated time

4.1.5 Order Tracking

- Real-time order status updates:
- Order Placed Accepted by Store Preparing Rider Assigned Out for Delivery Delivered
- Live map showing rider's current location as they approach
- Estimated time of arrival (ETA) display
- Push notification at each status change

4.1.6 Order History

- List of all past orders with date, store, items, total
- Repeat order functionality
- Order detail with itemized receipt
- Rating & review for store (post-delivery)

4.2 Store Admin Module

4.2.1 Authentication

- Dedicated login for Store Admins (role: `store_admin`)
- Each admin is linked to exactly one store
- Secure JWT with store-scoped access only

4.2.2 Store Setup & Management

- Create store profile: name, logo, banner image, category, description
- Set operating hours (per day of week, open/closed toggle)
- Toggle store open/closed status in real-time
- View store analytics: total orders, revenue, popular items

4.2.3 Product / Menu Management

- Create product categories (e.g., Burgers, Drinks, Sides)
- Add/edit/delete products:
- Name, description, price, image upload, category, availability
- Toggle product availability (in-stock / out-of-stock)
- Bulk product management

4.2.4 Order Management (Core Admin Flow)

- Real-time incoming order notifications (FCM push + in-app alert)
- Order queue view: pending orders listed with timer
- **Order status flow (admin-controlled):**

1. **New Order Received** Admin reviews items
2. **Accept / Reject** Admin taps Accept or Reject with reason
3. **Preparing** Admin marks food preparation started
4. **Ready for Pickup** Admin marks order ready for rider collection
5. **Out for Delivery** Rider picks up (auto-triggered or manual)
6. **Delivered** Order completed

- Order detail view: customer info, items, delivery address, payment type
- Order history and filtering by status/date

4.3 Rider Module

4.3.1 Authentication & Profile

- Rider registration: name, phone, vehicle type, license photo upload
- Login with rider credentials
- Profile: personal info, vehicle info, online/offline toggle
- Earnings summary dashboard (daily/weekly)

4.3.2 Order Assignment Flow

- Rider sets status to **Online** to receive orders
- Incoming order notification: store name, customer area, estimated distance
- **Accept / Reject order within a countdown timer** (e.g., 30 seconds)
- If rejected or timed out offered to next available rider
- Accepted order details: store address, items list, customer delivery address

4.3.3 Delivery Navigation

- **Google Maps integration:**
- Step 1: Navigate to store (restaurant pickup point)
- Step 2: Navigate to customer's delivery address
- Turn-by-turn directions within the app
- One-tap to open Google Maps or Apple Maps externally (optional)

4.3.4 Live Location Tracking (Critical Feature)

- Rider's GPS coordinates broadcast every 35 seconds to backend via WebSocket
- Customer app subscribes to rider location and renders live on map
- Admin and Super Admin can also view rider locations in real-time
- Background location tracking maintained when app is minimized

4.3.5 Delivery Completion

- Mark order as **Picked Up** at store
- Mark order as **Delivered** at customer door
- Optional: OTP-based delivery confirmation (customer provides 4-digit code to rider)
- Photo proof of delivery (optional enhancement)

4.3.6 Rider Earnings

- Per-delivery earnings tracked automatically
- Daily/weekly earnings summary
- Payout history (future: direct bank transfer integration)

4.4 Super Admin Module

4.4.1 Platform-Wide Dashboard

- Summary metrics:
- Total active orders right now
- Total registered customers, riders, store admins
- Total stores on platform
- Total revenue (daily/weekly/monthly)
- Total completed / cancelled orders
- Real-time activity feed

4.4.2 User Management

- **Customers:**
- View all registered customers
- Search by name, email, phone
- View customer order history
- Suspend / reactivate / delete account
- **Riders:**
- View all riders with online/offline status
- View rider's current location on map
- View rider's delivery history and earnings
- Approve new rider applications
- Suspend / reactivate riders
- **Store Admins:**
- Create new Store Admin accounts and link to a store
- Edit Store Admin details
- Suspend / reactivate store admins

4.4.3 Store Management

- Create / edit / delete stores on the platform
- View all stores with active/inactive status
- Monitor per-store order volume and revenue
- Toggle store visibility on the customer app

4.4.4 Order Management (Platform-Wide)

- View ALL orders across all stores with full detail
- Filter by: status, store, rider, date range
- Intervene in order disputes
- Issue refunds (triggers payment gateway refund API)

4.4.5 Analytics & Reporting

- Revenue charts: daily, weekly, monthly
- Top-performing stores
- Most ordered products
- Rider performance rankings
- Customer retention metrics
- Exportable reports (CSV/PDF)

---

5. Database Schema (High-Level)

```
-- Core Tables

users (id, name, email, phone, password_hash, role, is_active, created_at)
-- role: ENUM('customer', 'rider', 'store_admin', 'super_admin')

stores (id, admin_user_id FK, name, logo_url, banner_url, category,
description, is_active, is_open, operating_hours JSONB, created_at)

products (id, store_id FK, category_id FK, name, description, price,
image_url, is_available, created_at)

product_categories (id, store_id FK, name, sort_order)

customer_addresses (id, user_id FK, label, address_line, lat, lng, is_default)

orders (id, customer_id FK, store_id FK, rider_id FK NULLABLE,
status ENUM(...), payment_method ENUM('card','cod'),
payment_status ENUM('pending','paid','refunded'),
subtotal, delivery_fee, total, delivery_address JSONB,
created_at, updated_at)
-- status: ENUM('placed','accepted','rejected','preparing','ready',
-- 'picked_up','out_for_delivery','delivered','cancelled')

order_items (id, order_id FK, product_id FK, product_name, quantity,
unit_price, total_price)

order_status_logs (id, order_id FK, status, changed_by_user_id, timestamp)

payments (id, order_id FK, gateway, transaction_id, amount,
currency, status, created_at)

rider_locations (id, rider_id FK, lat, lng, timestamp)
-- Frequently written; consider time-series optimization or Firebase

rider_earnings (id, rider_id FK, order_id FK, amount, date)

notifications (id, user_id FK, title, body, type, is_read, created_at)

ratings (id, order_id FK, customer_id FK, store_id FK, rating, review, created_at)
```

---

6. API Design (Key Endpoints)

Authentication

```
POST /api/auth/register — Register (customer/rider)
POST /api/auth/login — Login, returns JWT
POST /api/auth/refresh — Refresh JWT token
POST /api/auth/logout — Invalidate token
POST /api/auth/forgot-password — Trigger OTP/email
POST /api/auth/reset-password — Reset with OTP
```

Customer Endpoints

```
GET /api/stores — List active stores (with pagination/filters)
GET /api/stores/:id — Store detail + categories + products
GET /api/stores/:id/products — Products of store
POST /api/orders — Place new order
GET /api/orders/my — Customer's order history
GET /api/orders/:id — Order detail + live status
GET /api/orders/:id/rider-location — Live rider lat/lng (polling or WS)
```

Store Admin Endpoints

```
GET /api/admin/store — Own store detail
PUT /api/admin/store — Update store profile/hours
POST /api/admin/products — Create product
PUT /api/admin/products/:id — Update product
DELETE /api/admin/products/:id — Delete product
GET /api/admin/orders — All orders for this store
PATCH /api/admin/orders/:id/status — Update order status
```

Rider Endpoints

```
PATCH /api/rider/status — Set online/offline
GET /api/rider/orders/available — See available orders to accept
PATCH /api/rider/orders/:id/accept — Accept order
PATCH /api/rider/orders/:id/reject — Reject order
PATCH /api/rider/orders/:id/status — Update: picked_up, delivered
POST /api/rider/location — Broadcast current GPS coords
GET /api/rider/earnings — View earnings
```

Super Admin Endpoints

```
GET /api/super/dashboard — Platform metrics
GET /api/super/users — All users (filterable by role)
PATCH /api/super/users/:id/status — Suspend/activate user
GET /api/super/stores — All stores
POST /api/super/stores — Create store + admin account
DELETE /api/super/stores/:id — Remove store
GET /api/super/orders — All orders platform-wide
POST /api/super/orders/:id/refund — Initiate refund
GET /api/super/analytics — Revenue, performance reports
```

---

7. Real-Time Features & Implementation

7.1 Order Status Updates

- **Technology:** Firebase Realtime Database or Socket.IO
- When an admin updates order status backend updates DB and emits event
- Customer app subscribes to `orders/{order_id}/status`
- Rider app subscribes to `orders/{order_id}` for acceptance confirmation

7.2 Rider Location Tracking

- **Technology:** Firebase Realtime Database
- Rider app writes GPS coordinates every 35 seconds to: `rider_locations/{rider_id}`
- Customer app reads from this node and updates map marker in real-time
- Super Admin panel subscribes to all rider locations for fleet view

7.3 Push Notifications (FCM)

Trigger	Recipient	Message
New order placed	Store Admin	"New order #123 received!"
Order accepted by store	Customer	"Your order is being prepared!"
Rider assigned	Customer	"Rider is on the way!"
Order status changed	Customer	"Your order is out for delivery!"
New order available	Riders	"New delivery order nearby!"
Order delivered	Store Admin	"Order #123 delivered successfully"

---

8. Third-Party Integrations

Integration	Purpose	Estimated Setup Time
Google Maps SDK	Maps display, rider tracking, address pin drop	3–4 days
Google Directions API	Turn-by-turn routing for rider	2 days
Google Places API	Address autocomplete in checkout	2 days
Firebase (Auth + FCM + Realtime DB + Storage)	Auth, notifications, real-time, file storage	3–5 days
Stripe / Braintree	Online card payment processing	4–5 days
Twilio / Firebase Phone Auth	OTP for phone verification	2 days

---

9. Security Considerations

- **JWT Authentication:** Short-lived access tokens (15 min) + refresh tokens (7 days) via `python-jose`
- **Role-Based Access Control (RBAC):** FastAPI `Depends()` guards on every route enforce role policy
- **Data Validation:** Pydantic models enforce strict input/output validation on all FastAPI endpoints
- **Password Security:** Passwords hashed with `bcrypt` via `passlib` never stored in plaintext
- **Payment Security:** Card data never stored on our servers; tokenized server-side via Stripe SDK for Python
- **HTTPS Enforcement:** TLS/SSL enforced at reverse proxy (Nginx) on all API traffic
- **Location Privacy:** Rider GPS coordinates only written to Firebase during active delivery; cleared post-delivery

- **Rate Limiting:** `slowapi` middleware for per-IP rate limiting on all FastAPI routes
- **CORS:** Strict CORS policy in FastAPI only whitelisted origins permitted
- **Environment Variables:** All secrets stored in `.env` + loaded via `python-dotenv`, never hardcoded

---

10. Deliverables & Timeline (12-Week Plan)

Phase 1 Foundation & Setup (Weeks 12)

Week	Deliverable	Details
Week 1	Project Setup & Architecture	Git repo, folder structure, CI/CD pipeline, dev/staging environments set up
Week 1	Backend Initialization	FastAPI project init, PostgreSQL schema with SQLAlchemy + Alembic, Firebase project setup
Week 1	Authentication System	Register, login, JWT (python-jose), bcrypt hashing, RBAC middleware for all 4 roles
Week 2	Store & Product Management APIs	CRUD APIs for stores, product categories, products — validated via Pydantic
Week 2	Customer App — Skeleton	Kotlin Android project init (Jetpack Compose), navigation structure, auth screens
Week 2	Store Admin App — Skeleton	Kotlin Auth screens, basic dashboard shell with Compose navigation

Milestone 1 Deliverable: Working auth for all roles + store/product management backend

---

Phase 2 Core Ordering Flow (Weeks 35)

Week	Deliverable	Details
Week 3	Customer — Store Listing & Browse	Store list screen, store detail, product browsing, categories
Week 3	Customer — Cart & Checkout UI	Cart management, checkout flow, address selection
Week 4	Payment Integration	Stripe Python SDK integration (server-side payment intent), Kotlin Stripe SDK (client-side), COD option
Week 4	Order Placement Backend	FastAPI order creation endpoint, order items, payment record creation via SQLAlchemy

Week	Deliverable	Details
Week 4	Store Admin — Order Management	Kotlin incoming orders screen, accept/reject Compose UI, PATCH status API
Week 5	Real-time Order Status	FastAPI WebSocket endpoint + Firebase Realtime DB for live order status sync to Kotlin app
Week 5	Push Notifications — Phase 1	FCM via firebase-admin Python SDK; notify store admin on new order, customer on status change

Milestone 2 Deliverable: Full end-to-end order flow working Customer places order Admin receives & manages Customer sees updates

---

Phase 3 Rider System & Maps (Weeks 68)

Week	Deliverable	Details
Week 6	Rider App — Core	Kotlin app: auth, Compose profile screen, online/offline toggle, available orders list
Week 6	Rider — Accept/Reject Flow	FCM notification to rider via firebase-admin (Python), countdown timer UI in Kotlin Compose
Week 7	Google Maps Integration	Google Maps SDK in Kotlin (customer + rider apps), Compose MapView, address pin drop
Week 7	Rider Navigation	Google Directions API called from FastAPI; route rendered in Kotlin app via Maps SDK
Week 8	Live Rider Location Tracking	Android FusedLocationProvider (Kotlin) → FastAPI WebSocket → Firebase → Customer Kotlin app map marker
Week 8	Rider Status Updates	Kotlin UI: Picked Up / Delivered buttons → FastAPI PATCH endpoint → FCM notification chain
Week 8	Push Notifications — Phase 2	firebase-admin (Python) sends FCM to riders for new orders; customer notified when rider en route

Milestone 3 Deliverable: Complete rider flow with live GPS tracking visible to customer on map

---

Phase 4 Super Admin & Analytics (Weeks 910)

Week	Deliverable	Details
Week 9	Super Admin Panel — UI	React.js web dashboard, Axios calls to FastAPI, JWT-protected routes
Week 9	Super Admin — User Management	FastAPI `/super/users` endpoints; React UI for view/search/suspend of all roles
Week 9	Super Admin — Store Management	Create stores + admin accounts via FastAPI; React store management table
Week 10	Super Admin — Order Oversight	Platform-wide order view via FastAPI paginated endpoint; refund trigger via Stripe Python SDK
Week 10	Analytics & Reports	SQLAlchemy aggregate queries for revenue, top stores, rider stats; CSV export via Python csv module
Week 10	Rider Earnings Tracking	Earnings calculated in FastAPI service layer; Kotlin Compose earnings dashboard for riders

Milestone 4 Deliverable: Fully functional Super Admin panel with complete oversight capability

---

Phase 5 Testing, Polish & Deployment (Weeks 1112)

Week	Deliverable	Details
Week 11	End-to-End Testing	Full user journey testing for all 4 roles
Week 11	Bug Fixes & Edge Cases	Handle payment failures, no riders available, store closed, etc.
Week 11	UI/UX Polish	Final UI refinements, loading states, error messages, empty states
Week 11	Performance Optimization	API response time optimization, query indexing, image caching
Week 12	Security Audit	Review all endpoints, check for vulnerabilities
Week 12	Staging Deployment	Full app deployed on staging server for client review
Week 12	Documentation	API docs (Postman collection), basic setup/deployment guide
Week 12	Final Delivery & Handover	Source code, APK builds, server credentials, deployment guide

Milestone 5 Deliverable: Fully tested, deployed, and documented application ready for launch

---

11. Team Structure & Roles

Role	Responsibility	Recommended Count
Python Backend Developer	FastAPI, SQLAlchemy, PostgreSQL, Firebase Admin SDK, Stripe, WebSocket	1–2
Kotlin Android Developer	Native Android (Jetpack Compose), Retrofit, Google Maps SDK, FCM, Coroutines	1–2
Frontend Developer	Super Admin web panel (React.js + Axios)	1
UI/UX Designer	App screens design (Figma), Compose UI flow design	1
QA Tester	End-to-end testing on real devices, API testing with Postman	1
Project Manager / Tech Lead	Coordination, client communication, code review, timeline tracking	1

---

12. Risk Assessment & Mitigation

Risk	Likelihood	Impact	Mitigation
Payment gateway approval delays	Medium	High	Apply for Stripe account in Week 1; use sandbox for development
Google Maps API quota limits	Low	Medium	Monitor usage; set quotas; cache geocoding results
Real-time location battery drain	Medium	Medium	Optimize GPS polling interval; use fused location provider
Rider location accuracy issues	Medium	Medium	Use Google Fused Location for best accuracy
Scope creep	High	High	Freeze feature scope in Week 1; document change requests separately
Payment disputes / refunds	Low	High	Implement clear refund logic early; test edge cases in payment flow
Push notification delivery failures	Low	Medium	Implement in-app notification fallback; retry logic
Timeline overrun on Maps integration	Medium	Medium	Allocate buffer week (Week 8 has flexibility)

---

13. Testing Strategy

13.1 Unit Testing

- **Backend (Python):** `pytest` + `pytest-asyncio` for FastAPI route and service layer tests
- **Android (Kotlin):** JUnit 4/5 + MockK for ViewModel and Repository unit tests
- Test coverage target: 70%+ on critical paths (auth, orders, payments)

13.2 Integration Testing

- **API:** `httpx` + `TestClient` (FastAPI) for full API integration tests against a test PostgreSQL database
- **Android:** Espresso + Compose UI testing for UI integration flows
- Payment flow tested in Stripe sandbox mode via Python test suite
- Firebase rules tested with Firebase local emulator suite

13.3 End-to-End Testing (Manual)

- Full user journey for each role documented as test cases
- Tested on real Android devices (multiple screen sizes)
- Order flow tested: place accept prepare rider pick up deliver

13.4 Performance Testing

- Load test APIs for concurrent orders (simulate 50+ simultaneous orders)
- Real-time location update stress test (20+ concurrent riders)

---

14. Deployment Architecture

```
Production Environment:

Internet ■■■ CloudFlare (CDN + DDoS protection)
■
▼
Load Balancer
■
■■■■■■■■■■■■■■■
▼ ▼
API Server 1 API Server 2 (AWS EC2 / DigitalOcean Droplets)
■
■■■■ PostgreSQL (Managed DB – AWS RDS or Supabase)
■■■■ Redis (session caching, rate limiting)
■■■■ Firebase (real-time DB, FCM, Storage)
■■■■ Stripe API (external payment)
```

---

15. Post-Launch Considerations (Future Phases)

These features are NOT in scope for the current 12-week delivery but are recommended for future development:

- iOS App (Kotlin Multiplatform Mobile can share business logic; UI would need Swift/SwiftUI)
- Customer loyalty program & promo codes / discount system
- Restaurant ratings & reviews with moderation
- Multiple language support (localization)
- Dark mode theme
- Advanced analytics with AI-based demand forecasting
- Rider incentive/bonus system
- Integration with accounting software (invoicing, tax reports)
- Customer re-order subscriptions / scheduled orders

16. Deliverables Summary

#	Deliverable	Tech	Format	ETA
1	Customer Android App	Kotlin + Jetpack Compose	Signed APK	Week 10
2	Rider Android App	Kotlin + Jetpack Compose	Signed APK	Week 10
3	Store Admin Android App	Kotlin + Jetpack Compose	Signed APK	Week 10
4	Super Admin Web Panel	React.js	Hosted Web App	Week 10
5	Backend REST API	Python + FastAPI	Deployed on Cloud Server	Week 10
6	Database Schema	PostgreSQL + SQLAlchemy	Production & Staging	Week 2
7	Firebase Project Setup	Firebase (Auth, FCM, Realtime DB, Storage)	Configured Project	Week 2
8	Payment Gateway Integration	Stripe Python SDK + Kotlin SDK	Live + Sandbox	Week 4
9	Google Maps Integration	Google Maps SDK (Android) + Directions API	Live in app	Week 7
10	API Documentation	FastAPI auto-generated OpenAPI (Swagger UI)	Interactive Docs	Week 12
11	Source Code	Git	GitHub Private Repo	Week 12
12	Deployment & Setup Guide	Markdown / PDF	Handover Docs	Week 12

17. Definition of Done

A feature is considered **Done** when:

- Backend API is implemented, tested, and returns correct responses
- Frontend (app/web) correctly integrates with the API
- Feature is tested on a real Android device
- Edge cases (errors, empty states, network failure) are handled
- Feature is deployed on staging and client can demo it
- Code is reviewed and merged to the main branch

This document is a living document and will be updated as requirements evolve. All changes must be formally agreed upon by both parties before implementation.

Document Version: 1.0 | **Confidential Development Team Use**